

Identity, Authorization & Observability

Detailed architecture and implementation specification

United States deployment baseline

IDENTITY	Entra users, managed identities, agents, federation and lifecycle governance
AUTHORIZATION	Layered RBAC, application policy, PermissionEnvelopes, decisions and revocation
TRACEABILITY	Durable linkage from principal and span to case, execution, evidence and ledger
AZURE	Application Insights, OpenTelemetry, Log Analytics, AMPLS, APIM and PostgreSQL

Planning baseline | Version 1.0 | 17 September 2026

Defines the shared control and proof fabric for foundation and KYC services. Configuration values are proposed and require target-subscription validation.

Contents

Purpose and outcome	4
1. Mission, boundary, and invariants	4
2. Logical architecture	5
3. Principal classes	6
4. Identity Registry	6
5. Human identity lifecycle	7
6. Workload and agent identity lifecycle	7
7. CI/CD workload federation	7
8. Authentication flows	8
9. Token validation standard	8
10. Authorization model	8
11. Policy decision contract	9
12. PermissionEnvelope	9
13. Decision precedence and failure posture	9
14. Authorization enforcement points	10
15. Separation of duties and privileged access	10
16. Identity and access events	11
17. Four linked records	11
18. Correlation identifiers	12
19. How traceability to a case is established	12
20. Correlation envelope	13
21. Trace topology	13
22. W3C propagation and asynchronous messaging	13
23. Span semantic conventions	14
24. Span events, status, and exceptions	14
25. Logs and structured events	14
26. Metrics	15
27. Sampling strategy	15
28. TraceLink persistence model	15
29. End-to-end case trace example	16
30. Authorization observability	16
31. Model, agent, and capability observability	16
32. Data protection for telemetry	16
33. Telemetry access and investigation	17
34. Azure Monitor topology	17
35. OpenTelemetry deployment standard	17
36. Alerting and SLOs	18
37. Cost and cardinality controls	18
38. KYC identity and traceability rules	18

39. Example forensic query path	19
40. APIs	19
41. Persistence model	20
42. Azure resource and identity configuration	20
43. CI/CD and policy as code	21
44. Implementation plan	21
45. Prioritized engineering backlog	22
46. Verification strategy	23
47. Operational runbooks	23
48. Architecture decisions and definition of done	23

Purpose and outcome

Identity, authorization, and observability form one control and proof fabric for the Harness Factory. Identity establishes who or what is acting. Authorization proves what that principal may do under a specific case, purpose, policy, and signed plan. Observability shows how a request moved through the platform. Durable correlation records connect operational spans to WorkItems, executions, commands, evidence, ledger events, and final dispositions without treating telemetry as the business source of truth.

This United States baseline extends the foundation, KYC domain, Harness Control Plane, Execution Control Plane, Agent Runtime and Capability Gateway, and Canonical Case Service specifications. It assumes Microsoft Entra ID, managed identities, workload identity federation, Azure API Management, Azure Monitor Application Insights with OpenTelemetry, Log Analytics, PostgreSQL correlation metadata, private networking, and immutable execution/ledger records.

1. Mission, boundary, and invariants

The identity and authorization plane governs human users, platform workloads, agents, adapters, automation, external integrations, and delegated users. The observability plane captures service behavior and correlated execution paths. They cooperate, but authentication logs and spans do not replace authorization decisions, ExecutionRecords, or the Case Ledger.

The following invariants are enforced:

- Every accepted action has an authenticated principal, actor class, tenant, audience, and authorization decision.
- Human, service, agent, gateway, adapter, deployment, and emergency identities remain distinct.
- Production workloads use managed identity or federated workload identity wherever supported; long-lived client secrets are exceptional.
- Authorization is evaluated from signed authority and current context, never from prompt text, agent self-assertion, network location, or trace attributes.
- The compiled PermissionEnvelope limits an agent even when its Entra identity has infrastructure reachability.
- Deny, revocation, expired authority, tenant mismatch, and stale fencing take precedence over allow.
- A trace ID is an operational correlation value. It is not the case ID, command ID, idempotency key, authorization proof, or audit record.
- Every material boundary propagates W3C Trace Context and a constrained correlation envelope.
- Durable records store trace references, while telemetry stores only approved opaque business references or hashes.
- Operational spans may be sampled. Business events, authorization decisions required for proof, ExecutionRecords, and Case Ledger events are never lost through telemetry sampling.
- Raw PII, credentials, tokens, prompts, documents, model output, and unrestricted tool payloads never appear in span attributes or ordinary logs.
- A privileged operator cannot both grant access and erase the evidence of that grant.

2. Logical architecture



Component	Responsibility	Source of truth
Microsoft Entra ID	Principal authentication, token issuance, groups, app roles, workload and agent identities	Directory identity and token claims
Identity Registry	Map Entra objects and agent/deployment identities to Harness actor IDs, owners, lifecycle, and risk	Harness actor metadata
APIM/auth middleware	Validate token, issuer, audience, signature, time, tenant, and required claims	Request authentication result
Authorization service	Evaluate action, resource, purpose, tenant, policy, assignment, compiled authority, and emergency state	AuthorizationDecision
PermissionEnvelope resolver	Resolve signed execution authority by digest	Runtime authority projection
OpenTelemetry SDK	Create and propagate spans, metrics, and structured log correlation	None; operational signal producer
OTel processor/exporter	Redact, enrich, sample, batch, and route telemetry	Pipeline configuration
Application Insights	Distributed trace, dependency, error, and performance analysis	Operational telemetry only
TraceLink service	Persist durable mapping among case/work/execution/command/event/evidence and traces	Correlation index
ExecutionRecord	Reproducible computational history	Forensic execution record
Case Ledger	Accepted business-state history	Business audit record

3. Principal classes

Principal class	Examples	Identity pattern
Human workforce	KYC analyst, supervisor, auditor, developer, SRE	Entra user, groups, app roles, Conditional Access, PIM for privileged roles
Platform workload	Workflow Kernel, Case Service, compiler, resolver, projector	Separate user-assigned or system-assigned managed identity per service
Agent	Screening, investigation, decision, QA agent	Dedicated Foundry/Entra agent identity or runtime deployment identity plus immutable AgentSpec
Gateway/adaptor	Capability Gateway, model gateway, screening adaptor	Separate managed identity per trust boundary and provider class
Automation	CI/CD, IaC deployment, security scanner	Federated workload identity tied to repository/workflow/environment
External system	Business application, partner integration	Entra application/service principal or federated identity with explicit audience and app role
Delegated user	Analyst action propagated to downstream API	OAuth on-behalf-of only where the downstream must apply user authority
Emergency	Break-glass administrator, emergency revoker	Dedicated cloud-only identity, strong controls, no routine use

One broad service principal for the platform is prohibited. Sharing identity across agents, services, tenants, or environments prevents least privilege, attribution, targeted revocation, and blast-radius control.

4. Identity Registry

The Identity Registry connects cloud identities to durable Harness actors. It does not replace Entra. It records ownership, actor type, environment, tenant scope, associated service/agent version, risk, approval status, credential mode, role assignments expected by IaC, and lifecycle.

```
create table identity.actor (
  actor_id uuid primary key,
  actor_type text not null,
  display_name text not null,
  environment text not null,
  tenant_scope jsonb not null,
  owner_group_id text not null,
  risk_class text not null,
  status text not null,
  created_at timestamptz not null,
  retired_at timestamptz
);

create table identity.principal_binding (
  binding_id uuid primary key,
  actor_id uuid not null references identity.actor(actor_id),
  entra_tenant_id text not null,
  entra_object_id text not null,
  application_id text,
  principal_kind text not null,
  foundry_agent_ref text,
  deployment_digest text,
  valid_from timestamptz not null,
  valid_to timestamptz,
  unique (entra_tenant_id, entra_object_id, valid_from)
);
```

A token proves the active Entra principal. The registry explains which Harness actor that principal represents at that time. Actor bindings are versioned; retirement closes validity rather than deleting history. Ledger and authorization records store both actor ID and token principal identifiers or protected hashes.

5. Human identity lifecycle

Human access follows joiner, mover, and leaver automation from the enterprise identity source. Business roles map to Entra groups and application roles; direct user assignments are exceptional. Case assignments and purpose remain application data because Azure resource RBAC cannot express KYC queue and case semantics by itself.

Analysts use phishing-resistant multifactor authentication where policy requires it, compliant device and location controls, and Conditional Access. Supervisors, policy owners, release managers, auditors, privacy/legal users, developers, and SREs have separate roles. Production diagnostics and administrative actions use PIM eligibility with justification, approval where configured, short activation, and access review.

Termination disables sign-in and group eligibility promptly. The platform invalidates sessions and refresh tokens according to risk. Open WorkItems are reassigned through business commands, preserving the former analyst as historical actor.

6. Workload and agent identity lifecycle

Azure workloads use managed identities because token issuance and rotation are platform-managed. Each service receives only the Azure data-plane and application roles needed for its function. Foundry-hosted agents use dedicated agent identity where available and still remain constrained by the compiled PermissionEnvelope.

The lifecycle is REQUESTED, PROVISIONED, ATTESTED, ACTIVE, SUSPENDED, REVOKED, and RETIRED. Provisioning is IaC. Activation requires owner, environment, purpose, role review, network placement, expected deployment digest, and monitoring. Drift between registry, Entra, Azure RBAC, Foundry, and IaC generates an alert.

Agent identity is not agent authorization. The same agent identity may reach the Capability Gateway, but the gateway authorizes a capability only for the signed plan, WorkItem, tenant, case, purpose, data class, provider binding, budget, and revocation generation supplied in the request.

7. CI/CD workload federation

CI/CD uses Entra workload identity federation rather than stored cloud credentials. Trust is constrained by repository or project, branch/tag or protected environment, workflow identity, issuer, subject, and audience. Production deployment has a distinct federated credential and role assignment from non-production.

Microsoft recommends managed identities for Azure-hosted workloads and workload identity federation for external workloads such as GitHub Actions or Kubernetes: [Authorize applications, resources, and workloads](#).

The deployment identity may deploy signed images and IaC but cannot approve Harness releases, read production cases, use KYC vendor tools, or sign release artifacts unless an explicit separate duty requires it. Pipeline traceability stores source commit, workflow run, federated subject, deployment operation, artifact digest, and resulting Azure resource changes.

8. Authentication flows

Scenario	Flow	Required checks
Analyst to Case API	Authorization code with PKCE	Tenant, issuer, audience, signature, time, app role/scope, Conditional Access context
Service to service	Managed identity client credential	Audience, object ID, app ID, actor binding, environment and role
Hosted agent to Capability Gateway	Dedicated agent/workload identity	Agent binding, deployment/version, audience, signed execution authority
CI/CD to Azure	Workload identity federation	Federated issuer/subject/audience and protected environment
Partner integration	Client credential or federation	Tenant allowlist, app role, certificate/federated binding, quotas
User-delegated downstream action	OAuth on-behalf-of	Original user authority plus service authority and downstream audience

The OAuth on-behalf-of flow is reserved for a middle tier calling a downstream API as the requesting user: [MSAL authentication flows](#). It is not used when application authority is sufficient. A service identity cannot silently impersonate a user, and delegated identity cannot expand the service or PermissionEnvelope authority.

9. Token validation standard

Every API validates token signature from trusted metadata, issuer, exact audience, tenant, nbf, iat, exp, allowed client/application, subject/object ID, and required scopes or app roles. Multi-tenant acceptance is explicit. Clock skew is bounded. Tokens are never logged or persisted in telemetry.

The edge produces an internal AuthenticationContext with actor binding, token principal, client application, tenant, authentication method/context where available, issued and expiry times, delegation status, and validation result. Downstream services receive a signed or integrity-protected form over mutually authenticated/private channels and reauthorize locally.

APIM validation is defense at ingress, not the only validation for internal services. High-value boundaries such as Case Command Gateway, Capability Gateway, Harness release, legal hold, and authorization administration validate independently.

10. Authorization model

Authorization combines several layers:

```
Authenticated principal
+ platform/application role
+ tenant membership and resource assignment
+ action and object type
+ purpose of use
+ case/work assignment
+ data classification and jurisdiction
+ signed PermissionEnvelope for machine execution
+ pinned policy snapshot
+ current revocation/emergency controls
+ environment, region, deadline, lease and fencing
= AuthorizationDecision + obligations
```

Azure RBAC controls Azure resource and data-plane reachability. Entra app roles and groups control coarse application roles. Azure ABAC conditions can refine supported Azure data actions, such as Blob access, but are not a general replacement for domain authorization: [Azure role assignment conditions](#). The Harness authorization service enforces case, purpose, policy, evidence, command, capability, and effect semantics.

11. Policy decision contract

```
{
  "decisionId": "authz_01...",
  "decisionVersion": "authorization.decision.v2",
  "principal": {
    "actorId": "actor_01...",
    "type": "AGENT",
    "entraTenantId": "...",
    "entraObjectIdHash": "sha256:..."
  },
  "request": {
    "action": "capability.invoke",
    "resource": "screening.sanctions.search@3.2.0",
    "tenantId": "tenant_01...",
    "caseId": "case_01...",
    "purpose": "KYC_SCREENING",
    "dataClasses": ["PII"],
    "effectClass": "READ_ONLY"
  },
  "authority": {
    "planDigest": "sha256:...",
    "permissionEnvelopeDigest": "sha256:...",
    "policyDigest": "sha256:...",
    "revocationGeneration": 4812
  },
  "outcome": "ALLOW",
  "reasonCodes": ["ROLE_ALLOWED", "PURPOSE_ALLOWED", "PROVIDER_ALLOWED"],
  "obligations": ["REDACT_TELEMETRY", "RETAIN_EVIDENCE_7Y"],
  "evaluatedAt": "...",
  "expiresAt": "...",
  "traceId": "..."
}
```

The durable record excludes raw token and payload. It stores policy and authority digests, protected principal reference, target, decision, reasons, and obligations. A span records the decision ID and outcome but not the full policy input.

12. PermissionEnvelope

The Harness compiler creates a `PermissionEnvelope` for each resolved `ExecutionProfile`. It contains case selectors, allowed commands, tools, skills, capabilities, data classes, purposes, regions, provider classes, egress, retention, budgets, effect classes, and explicit prohibitions.

The envelope is signed or included by digest in the signed plan. Runtime services retrieve it by digest, verify signature and release scope, check revocation, and intersect it with current policy and actor authority. They do not broaden it through local configuration.

Human users normally receive dynamic domain authorization from roles, assignments, queue, purpose, and policy rather than an agent `PermissionEnvelope`. High-risk human workflows can issue a short-lived access grant with similar explicit scope and obligations.

13. Decision precedence and failure posture

Evaluation order is designed for safe and explainable denial:

1. Invalid authentication, issuer, audience, tenant, or token time denies.
2. Revoked actor, release, plan, capability, provider, policy, or key denies.
3. Environment, tenant, region, case, assignment, deadline, lease, or fencing mismatch denies.
4. Explicit prohibition denies.
5. Missing required role, scope, purpose, data-class, capability, command, effect, or evidence authority denies.
6. Policy rule evaluates allow/deny and obligations.
7. Budget, rate, concurrency, and risk controls may deny or step up.

Authorization outages fail closed for mutations, restricted data, model/tool calls, and high-assurance work. Low-risk read-only views may use a short-lived previously evaluated decision only if policy explicitly permits it and revocation freshness remains inside the bound.

14. Authorization enforcement points

Boundary	Required enforcement
Control API	Authoring/release role, package scope, environment, separation of duties
Workflow Kernel	Plan, case, WorkItem, capability, assurance, lease, budget, revocation
Context Assembler	Case fields, evidence, purpose, data classes, tenant, trust partition
Agent Runtime	ExecutionProfile, PermissionEnvelope, allowed tools/commands, limits
Capability Gateway	Principal, capability/provider, purpose, data, effect, idempotency, region
Case Command Gateway	Principal, command, case, policy, evidence, concurrency, fencing
Evidence Store	Case/purpose/classification/retention/legal restriction and exact artifact
Human review UI/API	Queue/case assignment, role, authentication context, action, dual control
Observability query	Environment, dataset/table, incident/case purpose, field classification

Network controls and Azure resource roles support these boundaries but do not replace application decisions.

15. Separation of duties and privileged access

Duty	Required separation
Create identity	Cannot approve its production role assignment alone
Author policy	Cannot promote and waive its own failed evaluation
Release HarnessPlan	Cannot create evaluation attestation or alter signing key access
Operate Case Service	Cannot edit ledger or immutable audit export
Administer evidence storage	Cannot clear legal hold without legal workflow
Query production telemetry	Cannot automatically read raw case/evidence content
Investigate incident	Gets time-bound case/trace scope, not standing broad access
Emergency revoke	Can revoke rapidly but cannot create replacement authority alone

Use PIM, access reviews, approval, justification, time limits, alerts, and break-glass monitoring. Microsoft distinguishes Entra directory roles from Azure resource roles and supports review/governance of privileged assignments: [Microsoft Entra RBAC overview](#).

16. Identity and access events

Material identity and authorization changes produce governed events: actor provisioned/suspended/retired, binding changed, role assigned/removed, privilege activated, access review decided, policy released/revoked, emergency grant issued/expired, authorization denied at critical boundary, break-glass used, and drift detected.

Directory and Azure activity logs feed the security workspace and Sentinel integration. Application authorization decisions required for business proof are persisted by the authorization service. Case Ledger events reference actor and decision IDs for accepted business mutations.

Events contain opaque or hashed identity references where broad security distribution does not require full identity. Authorized investigators resolve them through the Identity Registry.

17. Four linked records

The platform preserves four records with different guarantees.

Record	Purpose	Sampling	Retention authority
OpenTelemetry	Performance, reliability, dependency, errors, capacity	Traces may be sampled; metrics not sampled	Observability policy
ExecutionRecord	Forensic computation: context, model, tools, output, usage	Never sampled	Execution/evidence policy
Case Ledger	Accepted material business transitions	Never sampled	Case retention/legal policy
ContextManifest	Exact governed information and trust partitions supplied to an agent	Never sampled for executed work	Execution/evidence policy

OpenTelemetry alone cannot explain a decision because spans can be sampled, transformed, or expired. The ExecutionRecord alone does not prove that a proposal changed business state. The Case Ledger records acceptance and links the command, authorization decision, execution, evidence, and trace.

18. Correlation identifiers

Identifier	Meaning	Created by	Durable locations
case_id	Canonical business aggregate	Case Service	Case, ledger, ExecutionRecord, restricted TraceLink
case_sequence	Ordered accepted state version	Case Service	Case and CaseEvent
work_item_id	Durable unit of work	Execution Control Plane	WorkItem, ExecutionRecord, TraceLink
execution_id	One execution attempt	Runtime dispatcher	ExecutionRecord, spans, TraceLink
command_id	One business intent submission	Caller/gateway	Command receipt, CaseEvent, TraceLink
authorization_decision_id	One policy evaluation	Authorization service	Decision store, span, command/capability record
evidence_id	Governed evidence object	Evidence service	Evidence metadata, lineage, CaseEvent
context_manifest_id	Governed model input set	Context Assembler	ContextManifest, ExecutionRecord
trace_id	One distributed operational trace	First instrumented boundary	Spans and TraceLink
span_id	One timed operation	Instrumented component	Span only, optional durable reference for key boundaries
correlation_id	Client-visible request support ID	Ingress	Response, logs, TraceLink; may map to trace ID or remain separate

Business identifiers are stable and authoritative in their own systems. The trace can be missing and the business record remains valid. The business record stores enough trace linkage to locate telemetry when present.

19. How traceability to a case is established

Traceability is built at the first case-aware boundary and reinforced at every material transition.

1. Ingress validates the caller, creates or accepts valid W3C traceparent, and assigns a correlation ID.
2. The Case Service resolves the opaque case ID and tenant under authorization.
3. A restricted TraceLink row binds case ID, trace ID, root span ID, correlation ID, actor, purpose, service, and time.
4. The root span receives `harness.case.ref`, normally a protected hash or opaque non-PII reference; exact case ID is limited to the restricted telemetry path.
5. WorkItem creation stores source CaseEvent ID and trace ID; the queued message carries traceparent, tracestate, and the minimal correlation envelope.
6. The worker creates a consumer span linked to the producer span and binds WorkItem and execution IDs.
7. ContextManifest, ExecutionRecord, capability invocations, authorization decisions, evidence objects, command receipts, and CaseEvents store their own durable IDs plus trace ID.
8. The final CaseEvent records command, actor, authorization decision, execution, evidence references, and trace ID.
9. The TraceLink index provides bidirectional queries from case/sequence/event to trace and from trace to business records.

This creates audit-grade lineage even if operational spans are sampled: the CaseEvent and ExecutionRecord remain durable, and the TraceLink reports telemetry availability.

20. Correlation envelope

The correlation envelope travels beside W3C trace context. It is integrity-protected within trusted services and contains only routing-safe identifiers.

```
{
  "version": "harness.correlation.v2",
  "tenantRef": "tn_h...",
  "caseRef": "case_h...",
  "caseSequence": 482,
  "workItemId": "wi_01...",
  "executionId": "exec_01...",
  "attempt": 2,
  "planDigestPrefix": "8f19c2a4",
  "purpose": "KYC_SCREENING",
  "classificationCeiling": "CONFIDENTIAL"
}
```

Do not put raw identifiers or authorization claims in W3C baggage because baggage propagates broadly and may reach third parties. The gateway strips internal headers before external provider calls and starts a sanitized dependency span. Provider correlation values are stored as protected hashes.

21. Trace topology

```
harness.case.operation [SERVER root]
  authentication.validate
  authorization.evaluate
  case.query / case.command
  workflow.transition
    workitem.create
    messaging.publish [PRODUCER]
    ... async boundary ...
    messaging.consume [CONSUMER, linked]
    execution.dispatch
    context.assemble
    agent.execute
      model.inference
      skill.execute
      capability.invoke
        authorization.evaluate
        provider.call
        evidence.persist
    result.validate
  case.command
    authorization.evaluate
    ledger.append
    outbox.write
```

A trace represents one operational causal path, not the entire lifetime of a case. A case can have thousands of traces. A long workflow uses links among traces through WorkItem, event, and execution IDs rather than keeping one multi-day trace open.

22. W3C propagation and asynchronous messaging

HTTP and gRPC propagate traceparent and tracestate according to W3C Trace Context. Services reject malformed values and create a new root when trust policy requires it. They never accept business authority from trace headers.

Service Bus messages carry trace context in application properties plus WorkItem ID, attempt, deadline, and correlation envelope. The send span is PRODUCER; the worker receive/process span is CONSUMER. Retries create new consumer spans linked to the same producer context and execution attempt. Dead-letter/replay adds links to original message and operator/replay operation.

Event Grid integration events carry trace context in CloudEvents extensions when supported. An outbox publisher creates a producer span linked to the database transaction's case-command span. A consumer processing the event creates its own trace or linked span according to retention and latency boundaries.

23. Span semantic conventions

Use standard OpenTelemetry attributes first, then a versioned `harness.*` namespace. High-cardinality business attributes are tightly controlled.

Span family	Required attributes
All server/worker	<code>service.name</code> , <code>service.version</code> , <code>deployment.environment.name</code> , <code>cloud.region</code> , <code>harness.operation.id</code>
Case	<code>harness.case.ref</code> , <code>harness.case.type</code> , <code>harness.case.sequence.before/after</code> , <code>command/event type</code>
WorkItem	<code>harness.work_item.id</code> , <code>type</code> , <code>attempt</code> , <code>priority</code> , <code>lease/fencing outcome</code>
Agent	<code>agent name/version/role</code> , <code>profile digest prefix</code> , <code>execution ID</code> , <code>outcome</code>
Context	<code>manifest ID</code> , <code>item counts by trust/classification</code> , <code>redaction count</code> , <code>token estimate</code>
Model	<code>logical profile</code> , <code>provider/deployment alias</code> , <code>input/output tokens</code> , <code>latency</code> , <code>finish class</code> , <code>cost estimate</code>
Capability	<code>capability/version</code> , <code>provider binding digest prefix</code> , <code>effect class</code> , <code>decision ID</code> , <code>status</code> , <code>billable units</code>
Authorization	<code>action</code> , <code>resource class</code> , <code>outcome</code> , <code>reason codes</code> , <code>policy digest prefix</code> , <code>cache status</code>
Case command	<code>command type</code> , <code>outcome</code> , <code>reason code</code> , <code>sequence before/after</code> , <code>event count</code>
Evidence	<code>evidence type</code> , <code>classification</code> , <code>size bucket</code> , <code>digest prefix</code> , <code>retention class</code> , <code>outcome</code>

Exact prompt, response, query, document, SSN, name, address, email, token, credential, evidence bytes, and policy-sensitive input are prohibited attributes. Digest prefixes identify versions but are not used as security proofs.

24. Span events, status, and exceptions

Use span events for meaningful milestones such as `budget.reserved`, `revocation.checked`, `policy.obligation.applied`, `provider.retry`, `evidence.registered`, `command.accepted`, and `ledger.committed`. Events carry bounded enums and counts.

Expected domain denials are recorded as normal authorization or command outcomes and may keep span status unset or OK with `harness.outcome=DENIED`. Infrastructure failures, malformed provider responses, unexpected exceptions, and violated platform invariants set error status. This distinction prevents legitimate denials from appearing as service failures while keeping them queryable.

Exception recording uses type, sanitized message, and stack trace according to environment and privacy policy. Exception messages are scrubbed because libraries may include URLs, queries, or identifiers.

25. Logs and structured events

Application logs are structured and correlate through trace and span context. Each event uses a stable event name and code, severity, service, environment, operation, outcome, and approved identifiers. Free-form string concatenation is avoided at controlled boundaries.

Logs do not duplicate span lifecycle noise. They record actionable state changes, warnings, retry exhaustion, drift, security signals, and operator actions. Audit-required authorization and business events go to their durable stores first; log export is secondary.

Security logs use a separate routing rule and workspace/table access model. Debug logging in production is time-bound, approved, sampled, and redacted. Changing log level cannot enable payload or token logging.

26. Metrics

Metrics provide unsampled service and control health. Key metric families include request rate/error/duration, queue age/depth, WorkItem state, authorization allows/denies by reason class, token validation failure, permission cache age, revocation generation lag, model tokens/cost/latency, capability/provider latency and circuit state, command conflicts, case sequence lag, outbox delay, evidence throughput, trace export failure, dropped spans/logs, and telemetry ingestion volume.

Labels use bounded dimensions. Case ID, WorkItem ID, execution ID, user ID, command ID, trace ID, URL, prompt, provider correlation, and free-form reason are prohibited metric labels.

Service-level objectives derive from metrics and authoritative counters where needed. Billing and decision counts do not rely exclusively on sampled spans.

27. Sampling strategy

Metrics are not sampled. Business records are not sampled. Traces use a coherent source sampling strategy so the same W3C trace sampling decision propagates across services. Microsoft recommends the Azure Monitor OpenTelemetry sampler and source sampling to preserve trace completeness; ingestion sampling is a fallback: [Application Insights OpenTelemetry sampling](#).

Initial policy keeps 100 percent of security-sensitive control-plane changes, critical authorization denials, high-risk effect attempts, canary traffic, and incident-scoped traces through explicit routing or durable security events. Routine successful high-volume reads can be sampled. All errors and selected slow traces should be retained through supported processor/routing patterns, validated per language and exporter.

Daily caps are last-resort cost controls because a cap creates an observability gap. Alerts fire well before the cap. A gap never removes business proof because authorization decisions, ExecutionRecords, command receipts, and CaseEvents are separate.

28. TraceLink persistence model

```
create table observability.trace_link (
  trace_link_id uuid primary key,
  tenant_id uuid not null,
  case_id uuid,
  case_sequence bigint,
  work_item_id uuid,
  execution_id uuid,
  command_id uuid,
  case_event_id uuid,
  evidence_id uuid,
  authorization_decision_id uuid,
  trace_id char(32) not null,
  root_span_id char(16),
  link_type text not null,
  service_name text not null,
  purpose text not null,
  telemetry_class text not null,
  observed_at timestamptz not null,
  expires_at timestamptz,
  unique (trace_id, link_type, service_name, observed_at)
);

create index ix_trace_link_case
  on observability.trace_link (tenant_id, case_id, case_sequence, observed_at);
create index ix_trace_link_execution
  on observability.trace_link (execution_id, observed_at);
create index ix_trace_link_command
  on observability.trace_link (command_id, observed_at);
```

The restricted TraceLink API enforces tenant, case, incident, and purpose authorization. It returns business links even when the telemetry retention window expired, marking the span as unavailable rather than implying the action did not occur.

29. End-to-end case trace example

For case_123, a sanctions refresh begins with CaseEvent evt_450 and WorkItem wi_987. The outbox publisher creates trace T1 and a producer span. A worker later consumes the message, links to T1, and creates execution exec_44. Context assembly creates ctx_22. The agent requests capability invocation cinv_81; authorization creates decision authz_66; the provider result creates Evidence ev_305. The agent proposes command cmd_91; the Case Command Gateway authorizes and commits CaseEvent evt_451 at sequence 483.

The durable chain is:

```
case_123@482 -> evt_450 -> wi_987 -> exec_44 -> ctx_22
             -> authz_66 -> cinv_81 -> ev_305 -> cmd_91
             -> evt_451 -> case_123@483
```

The operational trace shows timing and dependencies for the processing path. If the trace was sampled out or expired, the ExecutionRecord, authorization decision, evidence, command receipt, and ledger events still prove what occurred.

30. Authorization observability

Every authorization evaluation emits a low-cardinality metric and a span/event with decision ID, action class, resource class, outcome, reason codes, policy digest prefix, cache state, latency, and revocation generation. The full normalized input and decision are stored only when required in the authorization decision store.

Dashboards show deny rate by boundary and reason, missing/expired authority, tenant mismatch, stale fencing, revoked subject, policy evaluation errors, cache freshness, PIM activation, emergency access, and role drift. Alerts distinguish attacks, application defects, stale deployment, user entitlement issues, and dependency outages.

An unexpected allow is a critical incident. Reconciliation samples allowed actions and proves a matching actor binding, authority, decision, and downstream record. Test canaries continuously attempt prohibited actions and must be denied.

31. Model, agent, and capability observability

Model spans record logical profile, approved deployment alias, model API version, token counts, latency, finish class, safety outcome, retry/fallback, and estimated cost. They do not include prompt or response content. ExecutionRecord references the governed artifacts when retention permits.

Agent spans record AgentSpec and runtime version, ExecutionProfile digest, iteration count, model/tool call counts, budget consumption, stop reason, and result validation. Capability spans record capability version, provider binding digest, effect class, authorization decision, attempt, circuit, provider latency, evidence result, and normalized error class.

Fallback produces explicit child spans and a flight-recorder event. It cannot be hidden in an adapter. The trace and ExecutionRecord show why fallback was allowed and which evaluated provider produced the evidence.

32. Data protection for telemetry

Telemetry has a classification profile and attribute allowlist. Default classification is internal operational metadata. Exact case references, user object IDs, network details, stack traces, and security events use restricted tables/workspaces and narrower retention/access.

A telemetry processor drops or hashes prohibited attributes before export. It normalizes URLs to route templates, strips query strings and headers, scrubs exception messages, limits event size, and rejects dynamic attribute names. Automated tests inject seeded PII and secrets and assert none reaches exported telemetry.

Use Azure Monitor Private Link Scope to connect VNets privately to Application Insights and Log Analytics where required. Microsoft describes AMPLS as the private boundary for Azure Monitor resources and ingestion/query paths: [Azure Monitor Private Link](#).

33. Telemetry access and investigation

Access separates platform operations, application support, security operations, audit, privacy/legal, and engineering. Most operators see aggregate dashboards and sampled traces without exact case identity. Case-specific investigation requires a ticket/incident, purpose, time-bound role, and TraceLink authorization.

The investigator begins from an approved case, event, command, or execution ID. The TraceLink service returns trace IDs in scope. Azure Monitor queries retrieve spans. Artifact access remains a separate Evidence/Execution authorization and is not granted merely because a trace is visible.

Exports for regulators or litigation are assembled from Case Ledger, Evidence, ContextManifest, ExecutionRecord, AuthorizationDecision, Identity Registry, and available traces. The package labels telemetry as operational and identifies sampling/retention limits.

34. Azure Monitor topology

Use environment-separated Application Insights resources and Log Analytics workspaces, with additional security/audit workspaces or tables where access and retention require separation. Production and non-production never share an ingestion connection string or workspace.

Azure resource	Production baseline	Configuration
Application Insights	Workspace-based resource per production stamp or approved shared boundary	OTel ingestion, service map, availability, release annotations, local auth disabled where supported
Log Analytics	Production operations workspace plus security integration	Table-level retention/permissions, commitment tier after volume test, archive/export as required
AMPLS	Private monitoring boundary	Private endpoint, approved resources only, public access mode aligned to network policy
Data collection endpoints/rules	Custom logs and selected platform signals	Private link where needed, transformations, table routing, schema control
Azure Monitor Workspace	Prometheus metrics if platform requires it	Private query/ingestion design and Grafana authorization
Managed Grafana/Workbooks	Operational dashboards	Entra roles, no embedded broad credentials, parameterized case lookup through approved API
Storage/Event Hub export	Long-term security or audit routing	Private endpoint, CMK/WORM where required, separate identity
Sentinel	Security correlation	Identity, authorization, gateway, Defender, Key Vault, network, and anomaly detections

35. OpenTelemetry deployment standard

Services use the Azure Monitor OpenTelemetry distribution or supported exporter with one platform bootstrap package. The package configures resource attributes, propagation, sampler, redaction processor, exception policy, HTTP/database/messaging instrumentation, service version, environment, region, and exporter health.

Auto-instrumentation provides baseline HTTP, database, and messaging spans. Manual spans describe Harness semantics such as policy evaluation, context assembly, agent execution, evidence registration, case command, and ledger append. Instrumentation libraries are versioned and tested across languages.

Exporter failure is observable through local metrics/logs and does not fail business processing unless a high-risk workflow explicitly requires a durable proof record, which is written outside telemetry. Buffers are bounded. Telemetry backpressure cannot exhaust service memory or delay case commands indefinitely.

36. Alerting and SLOs

Objective	Proposed target	Alert
Token validation availability	99.99 percent at protected boundaries	Burn rate and any broad bypass symptom
Authorization service availability	99.95 percent monthly	Multi-window burn rate
Authorization latency	p95 under 40 ms cached; under 150 ms uncached	Sustained threshold by boundary
Revocation propagation	99.9 percent under 60 seconds	Any production component stale over 60 seconds
Trace propagation	99.5 percent of sampled internal requests retain parentage/link	Orphan ratio over threshold
Critical durable correlation	100 percent accepted commands have decision/execution/trace link where applicable	Any reconciliation gap
Telemetry PII leakage	Zero known prohibited values	Any DLP canary match
Export health	99.9 percent batches exported inside five minutes	Queue/drop/export error threshold
Clock synchronization	Maximum skew within approved bound	Host/service skew alert

Page immediately on unexpected authorization allow, break-glass use, privilege escalation drift, disabled token validation, revocation staleness, telemetry secret/PII canary, missing ledger-decision linkage, or systemic trace propagation break.

37. Cost and cardinality controls

Attribute and metric cardinality budgets are part of service readiness. Case, WorkItem, execution, command, trace, user, and evidence identifiers are not metric dimensions. Span retention and sampling vary by environment and risk. Large stack traces, exception bursts, dependency URLs, and repeated retries are bounded.

Budgets are monitored by service, environment, table, signal type, and release. Sudden ingestion growth creates an engineering incident before daily cap. Data collection transformations can drop unapproved fields, but source instrumentation remains the primary control.

Retention tiers distinguish hot operational analysis, longer security investigation, and durable business/forensic records. Moving all spans to long-term storage is not a substitute for an ExecutionRecord and creates unnecessary privacy and cost exposure.

38. KYC identity and traceability rules

KYC analysts have roles for intake, review, enhanced due diligence, disposition approval, quality assurance, audit, and supervision. Queue/case assignment and jurisdiction are checked in the application. High-risk overrides and selected dispositions require step-up or dual control.

Agent identities are separate for screening, investigation, decision recommendation, and QA where deployment permits. Their PermissionEnvelopes differ. A screening agent cannot acquire decision or evidence-download authority merely because it shares a Foundry project.

For each final disposition, the audit query must return:

- Effective CaseEvent and case sequence.
- Human/system actor and Entra/Harness identity binding valid at the action time.
- AuthorizationDecision, policy snapshot, and reason/obligations.
- DecisionRecommendation and DecisionGateResult.
- WorkItems and executions that produced relevant findings.

- ContextManifests, model/tool profiles, capability calls, and evidence lineage.
- Commands accepted or rejected and concurrency/idempotency results.
- Available operational traces and declared sampling/retention status.

39. Example forensic query path

An auditor starts with final disposition `disp_77`. The Case Ledger returns `evt_900`, sequence 612, command `cmd_812`, actor `actor_42`, decision `authz_702`, policy digest, recommendation, gate result, evidence references, execution IDs, and trace IDs.

Identity Registry resolves `actor_42` to the Entra principal binding that was valid at the event time. Authorization store returns the evaluated action, resource, purpose, role, `PermissionEnvelope` where applicable, outcome, reason codes, obligations, and revocation generation. `ExecutionRecords` resolve `ContextManifests`, model/tool versions, results, and usage. Evidence lineage returns the source chain. `TraceLink` returns Application Insights trace IDs for timing and dependency analysis.

Each hop verifies tenant, case, identifier, digest, sequence, and temporal validity. Missing sampled telemetry is reported as `TELEMETRY_NOT_RETAINED`; it does not invalidate the durable audit chain.

40. APIs

Endpoint	Purpose	Authorization
GET <code>/v1/actors/{id}</code>	Resolve actor metadata and historical bindings	Identity auditor or authorized service
POST <code>/v1/authorization:check</code>	Evaluate one action/resource/context	Authenticated service with allowed policy domain
POST <code>/v1/authorization:batchCheck</code>	Evaluate bounded field/object set	Strict item and size limits; same tenant/purpose
GET <code>/v1/authorization/decisions/{id}</code>	Read normalized decision	Resource/case/security purpose
POST <code>/v1/access-grants</code>	Issue short-lived scoped human grant	Approved privileged workflow
POST <code>/v1/revocations</code>	Revoke actor/grant/authority	Emergency revoker or governed admin
POST <code>/v1/trace-links</code>	Register durable business-to-trace relationship	Trusted service identity only
GET <code>/v1/cases/{id}/trace-links</code>	Find trace links for case/sequence/time	Case-specific diagnostic purpose
GET <code>/v1/traces/{traceId}/business-links</code>	Resolve governed business records	Restricted diagnostic/audit role
POST <code>/v1/audit-packages</code>	Assemble cross-record proof package	Audit/legal workflow and case scope

Authorization check requests require idempotent decision correlation but are not cached solely by caller. Cache keys include principal binding, action, resource, tenant, purpose, authority digests, policy, revocation generation, and relevant context version.

41. Persistence model

```
create table authorization.decision (
  decision_id uuid primary key,
  actor_id uuid not null,
  action text not null,
  resource_type text not null,
  resource_ref_hash text not null,
  tenant_id uuid not null,
  case_id uuid,
  purpose text not null,
  outcome text not null,
  reason_codes text[] not null,
  obligations jsonb not null,
  policy_digest text not null,
  permission_envelope_digest text,
  revocation_generation bigint not null,
  trace_id char(32),
  evaluated_at timestamptz not null,
  expires_at timestamptz
);

create table authorization.access_grant (
  grant_id uuid primary key,
  actor_id uuid not null,
  scope jsonb not null,
  purpose text not null,
  status text not null,
  issued_by uuid not null,
  approved_by uuid,
  issued_at timestamptz not null,
  expires_at timestamptz not null,
  revoked_at timestamptz
);
```

High-volume read decisions may use shorter durable retention or aggregated evidence according to risk. Mutation, capability effect, evidence access, disposition, release, legal hold, and privileged decisions retain full normalized records for their governing period.

42. Azure resource and identity configuration

Resource	Identity configuration	Observability configuration
APIM	Managed identity for backends; Entra JWT validation; admin roles separated	Application Insights logger, payload logging disabled, correlation headers controlled
Container Apps	User-assigned identity per service/trust boundary	OTel distro/exporter, environment/service version attributes, bounded buffers
Foundry agents	Dedicated agent identity and project boundary	Injected/explicit OTel integration, execution ID and profile correlation
Service Bus	Data sender/receiver roles per producer/consumer	Messaging spans, queue metrics, diagnostic settings, DLQ alerts
PostgreSQL	Entra managed identities and database roles	Connection/query metrics, sanitized dependency spans, audit configuration
Blob Storage	Managed identity and narrowly scoped data roles/ABAC where supported	Request/dependency metrics, storage diagnostics, no object content
Key Vault	Managed identity and per-purpose RBAC	Access logs to security workspace, alert on unusual sign/secret operations
App Configuration	Managed identity; release projector write, consumers read	Refresh/failure metrics and configuration version attributes
Azure Monitor	Entra roles, AMPLS, local auth disabled where supported	Separate prod workspace/resource, DCRs, transformations, retention/export

43. CI/CD and policy as code

Identity definitions, federated credentials, role assignments, custom roles, APIM policies, service identities, AMPLS, Application Insights, Log Analytics, DCRs, alerts, workbooks, and diagnostic settings are IaC. Application roles and authorization policies are versioned with tests.

```
pull request
-> identity/role/policy lint
-> least-privilege and separation-of-duty tests
-> OTel semantic and redaction tests
-> trace propagation integration tests
-> IaC security + what-if/plan
-> deploy nonprod through federation
-> authorization conformance and negative tests
-> PII/secret telemetry canary
-> load, sampling, retention and cost tests
-> production approval and deploy by digest
-> post-deploy identity/role/diagnostic reconciliation
```

Policy changes include semantic diff: new principals, roles, actions, resources, data classes, purposes, effects, regions, or allow rules. Authority expansion requires additional approval and evaluation.

44. Implementation plan

Phase	Duration	Deliverables and exit criteria
0. Taxonomy and threat model	2 weeks	Principal classes, action/resource model, correlation semantics, data classification, threat model, ADRs
1. Entra and identity foundation	3 weeks	App registrations, managed identities, groups/app roles, federation, PIM, Conditional Access, registry
2. Authorization service	5 weeks	Decision API, policy engine, PermissionEnvelope resolution, cache, revocation, decision store
3. Enforcement integration	5 weeks	APIM, control, workflow, context, runtime, capability, case, evidence, human-review boundaries
4. OTel platform	4 weeks	Bootstrap library, semantic conventions, redaction, propagation, exporters, App Insights and workspaces
5. Durable traceability	4 weeks	TraceLink service, ExecutionRecord/CaseEvent links, case/trace queries, audit package assembly
6. Dashboards and alerts	3 weeks	SLOs, identity/access dashboards, trace quality, cost, security and KYC views
7. KYC pilot	4 weeks	Analyst/agent identities, end-to-end authorization and disposition traceability, adversarial tests
8. Hardening and operations	3 weeks	Load, outage, PII canary, break-glass, access review, incident and recovery exercises

45. Prioritized engineering backlog

ID	Work item	Acceptance criterion
IA-01	Define actor and principal taxonomy	Every human/workload/agent path maps to one versioned actor binding
IA-02	Provision managed identities and roles through IaC	No shared broad production principal; drift is detected
IA-03	Implement CI/CD federation	Production deploys without stored client secret and with protected subject constraints
IA-04	Build Identity Registry	Historical actor-to-Entra/agent/deployment binding resolves at event time
AZ-01	Define action/resource/purpose schema	All platform boundaries use stable, versioned authorization vocabulary
AZ-02	Implement decision service	Exact input produces explainable outcome, reasons, obligations, and durable decision ID
AZ-03	Integrate PermissionEnvelope and revocation	Machine authority cannot exceed signed plan and is fenced under 60 seconds
AZ-04	Implement human case/queue authorization	Tenant, assignment, role, purpose, jurisdiction, and field access are enforced
AZ-05	Add PIM/access review/emergency controls	Privileged use is time-bound, reviewed, alerted, and correlated
OB-01	Publish Harness OTel semantic convention	Cross-language fixtures emit the same required attributes and span topology
OB-02	Implement redaction processor	Seeded PII, tokens, secrets, payloads, and query strings never export
OB-03	Propagate W3C context over HTTP and messaging	Sampled traces maintain parentage/links across retries and outbox paths
OB-04	Build TraceLink service	Case/event/work/execution/command/decision/evidence resolve bidirectionally to traces
OB-05	Integrate durable records	Accepted CaseEvent links actor, decision, execution, evidence, command, and trace
OB-06	Configure Azure Monitor private topology	Production ingestion/query uses approved AMPLS/workspace access design
OB-07	Implement sampling and cost policy	Trace completeness and critical retention meet targets within budget
SEC-01	Build unexpected-allow and PII canaries	Any accepted prohibited action or exported seeded secret pages immediately
KYC-01	Produce disposition audit package	One query reconstructs identity, authority, computation, evidence, ledger, and telemetry links

46. Verification strategy

Unit tests cover token claim validation, actor binding, role/action/resource mapping, policy precedence, PermissionEnvelope intersection, revocation, decision reasons/obligations, cache keys, correlation headers, attribute allowlists, redaction, and TraceLink constraints.

Contract tests run for every enforcement point and identity class. Negative cases include wrong audience, tenant, issuer, expired token, unbound principal, suspended actor, missing role, wrong purpose, cross-case access, stale plan, revoked capability, stale fencing token, stronger effect, and unauthorized telemetry query.

Trace tests exercise synchronous HTTP, nested services, Service Bus send/receive, duplicate delivery, retries, dead-letter/replay, outbox publishing, model/tool dependencies, and provider header stripping. They verify parentage, links, execution attempts, durable record references, and sampled-trace behavior.

Data-loss tests sample out traces, expire Log Analytics data, fail exporter, reach a test cap, and block Azure Monitor. Case Ledger, ExecutionRecord, ContextManifest, AuthorizationDecision, and TraceLink still reconstruct the business path and label missing telemetry accurately.

Privacy tests seed SSNs, names, emails, tokens, API keys, prompts, documents, query strings, and provider payloads into all inputs and exceptions. No prohibited value may appear in exported spans, logs, metrics, dashboards, alerts, or workbooks.

47. Operational runbooks

Required runbooks cover token validation outage, Entra metadata/key refresh, managed identity failure, federated credential failure, unexpected authorization allow, deny spike, authorization latency/cache failure, revocation lag, PIM/break-glass use, role drift, compromised principal, trace propagation break, exporter backlog, dropped spans, Application Insights outage, AMPLS/DNS failure, ingestion spike/cap risk, PII/secret canary, missing TraceLink, and audit-package inconsistency.

Each runbook states detection, impacted boundaries, immediate containment, identity or policy revocation, safe cache behavior, diagnostic queries, evidence preservation, recovery, communication, and verification. Operators never bypass authorization to restore service or enable payload logging as a first diagnostic step.

48. Architecture decisions and definition of done

The initial decisions approve Entra as identity backbone; one identity per service/agent trust boundary; managed identity and workload federation as preferred credentials; application authorization layered above Azure RBAC; PermissionEnvelope as machine-execution authority; deny/revocation precedence; W3C Trace Context; versioned Harness OTel conventions; a restricted TraceLink index; and separate OpenTelemetry, ExecutionRecord, ContextManifest, and Case Ledger records.

Production pilot readiness requires all of the following:

- Every human, workload, agent, gateway, adapter, and automation path has a unique governed identity and historical actor binding.
- No production service requires a long-lived client secret where managed identity or federation is supported.
- Every protected action produces an explainable decision with actor, action, resource, tenant, purpose, policy, authority, outcome, reasons, and obligations.
- Agent authority cannot exceed the signed PermissionEnvelope even when infrastructure access exists.
- Revocation reaches enforcement points in under 60 seconds and stale authority fails closed.
- W3C context propagates across HTTP, messaging, retries, outbox, agent, model, capability, case command, and ledger boundaries.
- Every accepted case mutation links the CaseEvent to actor, authorization decision, command, execution, evidence, and trace where applicable.
- A final disposition can be reconstructed from case and durable proof records without relying on retained sampled spans.

- Telemetry contains no seeded prohibited PII, secret, token, prompt, document, or provider payload.
- SLOs, sampling, AMPLS, access controls, retention, cost alerts, break-glass, incident response, and audit package generation are tested.
- Security, IAM, privacy, KYC policy, audit, platform, SRE, and operations owners approve the evidence and ownership model.

The result is a platform where every consequential action has a known actor, bounded authority, explainable decision, durable business lineage, and an operational trace that can be followed from request to case outcome without confusing telemetry with truth.